WINDOWS SERVER ADMINISTRATION LAB

Recurring Domain Account Lockout

Tracing a Stale Saved Credential to Its Source

A practical troubleshooting report: problem, investigation, fix and prevention

Prepared by
Urvisha Bhoi

Environment: VMware lab, testuser.local domain (DC01 + WIN11-WS01)

Contents

Report Overview The ticket, what I did, lab environment

Part A: Building the Lab Static IP, domain controller, test user, domain join

Part B: Preparing the Lab Lockout policy, auditing, Task Scheduler history

Part C: Planting the Problem Scheduled task with a saved password, password changes

Part D: The Investigation Events 4740, 4771, 4625 and finding the exact task

Part E: The Fix Fixing in the right order and final verification

Part F: Automating the Investigation Find-LockoutSource.ps1

Incident Timeline

Command Reference Every command with where it ran and what it does

What I Learned

Summary and Security Recommendations

Report Overview

The ticket

> **USER REPORT**
> "IT User01 keeps getting locked out every few minutes, even right after the help desk unlocks the account. The user says they have not typed a wrong password."

This is one of the most common escalated help desk tickets. Most of the time the user is telling the truth: something else is still using their **old password**. Common sources are a phone syncing email, a mapped drive, a Windows service, a saved credential, or a **scheduled task**. Every time that thing tries the old password, the domain controller counts a bad attempt, and after enough attempts it locks the account.

Unlocking the account only treats the symptom. The real job is to find **which computer** is sending the bad password and **what on that computer** is doing it, then fix it in the right order.

What I did in this project

1. Built a small Active Directory lab: one domain controller and one Windows 11 workstation.
2. Configured an account lockout policy and turned on the audit events needed to investigate lockouts.
3. Planted a realistic problem: a scheduled task that runs as IT User01 with a saved password that later became out of date.
4. Investigated the lockout like a help desk technician, using Active Directory, Event Viewer and PowerShell.
5. Fixed the source first, then unlocked the account, and verified the fix.
6. Wrote a PowerShell script (Find-LockoutSource.ps1) that performs the investigation in one command.

Lab environment

Item	Value
Virtualization	VMware Workstation, NAT network 192.168.76.0/24 (gateway 192.168.76.2)
Domain	**testuser.local** (NetBIOS name **TESTUSER**)
Domain controller	**DC01**, Windows Server 2022, static IP **192.168.76.138**, AD DS + DNS, PDC Emulator
Workstation	**WIN11-WS01**, Windows 11 Pro, static IP **192.168.76.137**, DNS pointing to DC01
Domain user	**IT User01** (logon name `ituser01`) in the OU **Lab Users**
Local admin on workstation	**Urvisha** (signed in as `.\Urvisha`)
Time zone (both VMs)	Mountain Time (UTC-07:00)

RESULT IN ONE SENTENCE

The lockouts were caused by the scheduled task "OneDrive Backup Sync" on WIN11-WS01, which was still using IT User01's old password. I found it through Event 4740 (source computer) and Event 4625 (Logon Type 4 = scheduled task), stopped it, unlocked the account, updated the task's saved password, and confirmed no further lockouts.

Part A: Building the Lab

Before troubleshooting, I needed a working domain. These steps turn a fresh Windows Server into a domain controller and join a Windows 11 PC to it.

A1. Give the server a fixed IP address and DNS

Both VMs first received addresses from VMware's DHCP service, which can change after a restart. A domain controller's IP address must never change, because every computer finds the domain by asking the DC's DNS. I made the server's address static and pointed its DNS at itself.

Run on: DC01 (Administrator Command Prompt)

```
netsh interface ipv4 set address name="Ethernet0" static 192.168.76.138 255.255.255.0
192.168.76.2
```

What it does: Sets a fixed IP address (192.168.76.138), subnet mask (255.255.255.0) and default gateway (192.168.76.2, the VMware NAT router) on the network adapter named Ethernet0.

Run on: DC01

```
netsh interface ipv4 set dnsservers name="Ethernet0" static 192.168.76.138 primary
```

What it does: Points the server's DNS to itself. Windows warned "The configured DNS server is incorrect or does not exist" because the DNS role was not installed yet. That warning is expected at this stage and the setting was still saved.

Run on: DC01

```
ipconfig /all
```

What it does: Shows the network settings. I used it to confirm DHCP Enabled: No, IPv4 Address 192.168.76.138 and DNS Servers 192.168.76.138.

Run on: DC01

```
powershell Rename-Computer -NewName DC01 -Restart
```

What it does: Renames the server from its random default name (WIN-5DSPUJ96AV3) to DC01 and restarts it. A clear name makes event logs much easier to read.

A2. Install Active Directory and create the domain

I used Server Manager (graphical method):

1. **Manage > Add Roles and Features**, selected **Active Directory Domain Services**, clicked **Add Features**, then **Install**.
2. Clicked **Promote this server to a domain controller**, chose **Add a new forest**, and entered the root domain name **testuser.local**.

3. Kept **DNS server** and **Global Catalog** ticked, set a Directory Services Restore Mode (DSRM) password, accepted the NetBIOS name **TESTUSER**, and clicked **Install**. The server restarted as a domain controller.

A3. Create the test user

In **Active Directory Users and Computers (ADUC)** I created an organizational unit named **Lab Users** and a user **IT User01** with logon name **ituser01**, and unticked "User must change password at next logon".

A4. Join the Windows 11 PC to the domain

1. Installed Windows 11 **Pro** (Home editions cannot join a domain) and created the local account **Urvisha** during setup.
2. In **ncpa.cpl** set a static IP **192.168.76.137**, mask 255.255.255.0, gateway 192.168.76.2 and **Preferred DNS 192.168.76.138** (the DC). DNS pointing to the DC is the most important setting for a domain join.
3. In **sysdm.cpl > Change...** renamed the PC to **WIN11-WS01**, selected **Domain**, entered **testuser.local**, and signed in with the domain Administrator account. Windows showed "Welcome to the testuser.local domain" and the PC restarted.

> **NOTE**
> I originally tested the domain with `nslookup testserver.local`, which failed. The welcome message showed the real domain name was **testuser.local**. Reading the actual name from the system, instead of assuming it, fixed the confusion.

Part B: Preparing the Lab for the Investigation

B1. Check the starting state

Before changing anything, I recorded how the domain looked. This gives a "before" picture for comparison.

Run on: DC01 (PowerShell as Administrator)

```
Get-ADDomain | Select-Object DNSRoot, NetBIOSName, PDCEmulator
```

What it does: Shows the domain name, short name and which DC holds the **PDC Emulator** role. The PDC Emulator is important because every account lockout is processed there, so it is where Event 4740 is recorded.

Run on: DC01

```
Get-ADUser ituser01 -Properties LockedOut, BadLogonCount | Select-Object SamAccountName,
LockedOut, BadLogonCount
```

What it does: Shows whether IT User01 is locked (LockedOut) and how many wrong passwords have been counted (BadLogonCount). At the start it was False and 0.

Run on: DC01

```
net accounts /domain
```

What it does: Displays the domain's password and lockout policy.

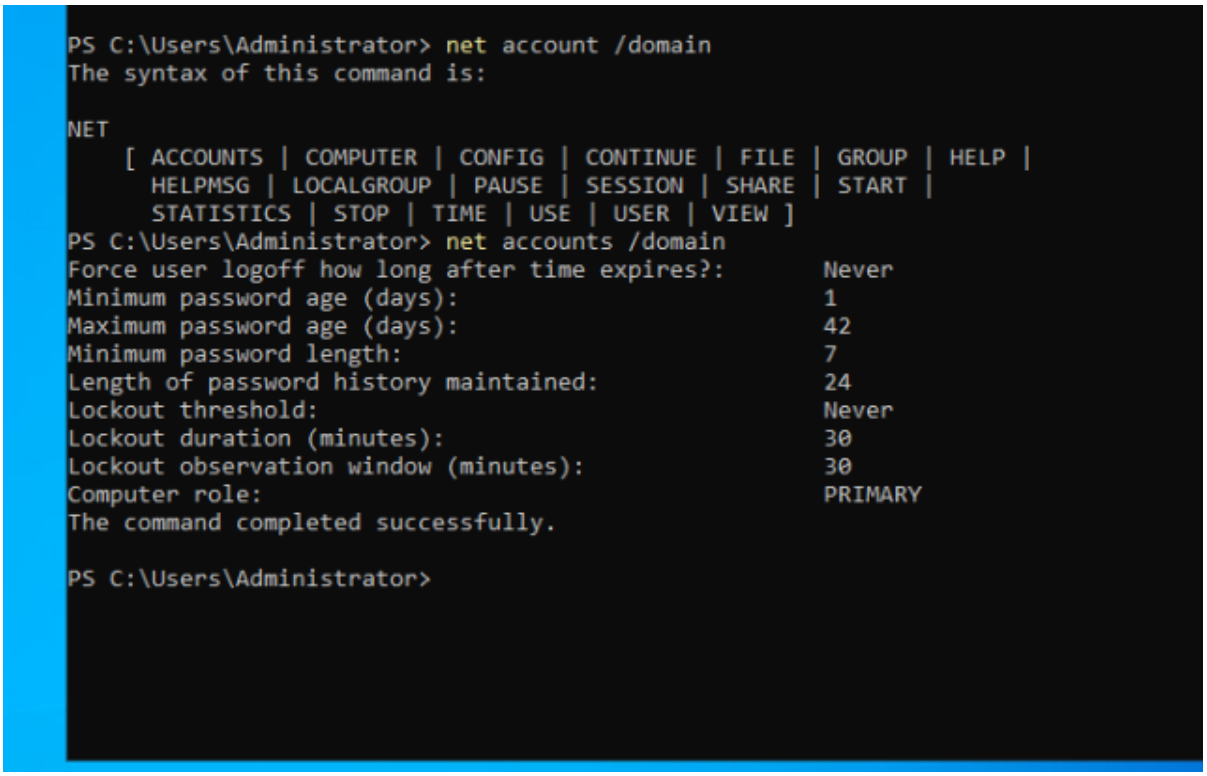


Figure 1. *Before: the domain had no lockout policy (Lockout threshold: Never). Without a threshold, accounts never lock, so this had to be configured first. (The first line shows a typo, "net account", which Windows rejected with its syntax help.)*

B2. Configure the account lockout policy

In **Group Policy Management** I edited the **Default Domain Policy** at: Computer Configuration > Policies > Windows Settings > Security Settings > Account Policies > **Account Lockout Policy**.

Setting	Value	Meaning
Account lockout threshold	5	After 5 wrong passwords the account is locked
Account lockout duration	15 minutes	The account unlocks by itself after 15 minutes (or when the help desk unlocks it)
Reset account lockout counter after	15 minutes	If no wrong password is entered for 15 minutes, the counter goes back to 0

> **WHY THE DEFAULT DOMAIN POLICY?**
> For domain accounts, Windows only reads password and lockout rules from a GPO linked at the top of the domain. The same settings in a GPO linked to an OU (such as Lab Users) are ignored for domain users.

Run on: DC01

```
gpupdate /force
net accounts /domain
```

What it does: `gpupdate /force` applies Group Policy immediately instead of waiting for the next refresh. Running `net accounts /domain` again confirms the new values.

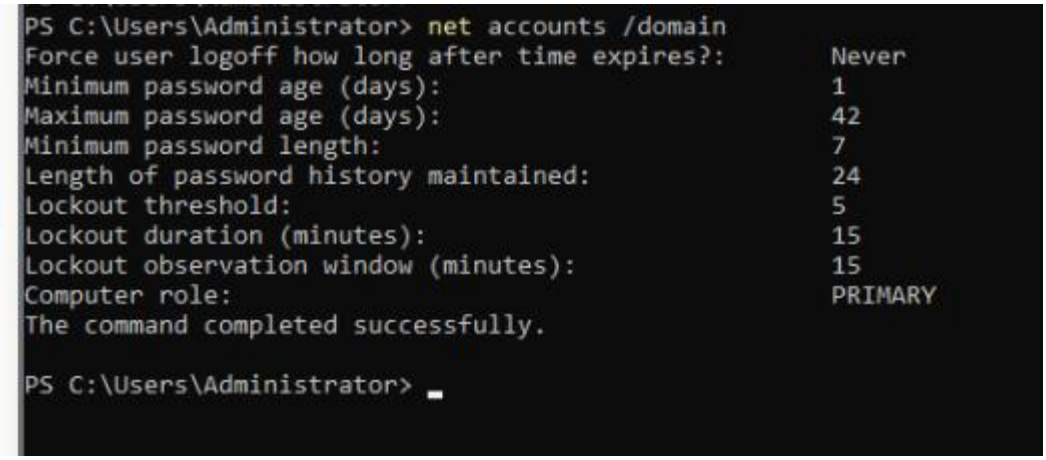


Figure 2. *After: Lockout threshold 5, lockout duration 15 minutes, observation window 15 minutes.*

B3. Turn on auditing on the domain controller

Auditing works like security cameras: it makes Windows write an event every time a lockout or failed sign-in happens. In **Group Policy Management** I edited the **Default Domain Controllers Policy** at: Computer Configuration > Policies > Windows Settings > Security Settings > **Advanced Audit Policy Configuration > Audit Policies**, and set the following to **Success and Failure**:

Audit subcategory	Event it creates	What the event records
Account Management > Audit User Account Management	**4740**	An account was locked out, including the **Caller Computer Name** (the source)
Account Logon > Audit Kerberos Authentication Service	**4771**	A Kerberos sign-in failed, with the **client IP** and a **failure code**
Account Logon > Audit Credential Validation	**4776**	An NTLM credential check (the older sign-in method)

> **A DETAIL MANY PEOPLE MISS**
>
> Event 4740 is logged as a **Success** event ("the lockout succeeded"). If only Failure is ticked for User Account Management, no lockout events are recorded.

Run on: DC01

```
gpupdate /force
auditpol /get /subcategory:"User Account Management"
auditpol /get /subcategory:"Kerberos Authentication Service"
auditpol /get /subcategory:"Credential Validation"
```

What it does: `auditpol /get` shows the audit setting that is actually in effect on the machine. This confirms the Group Policy was applied, not just saved.

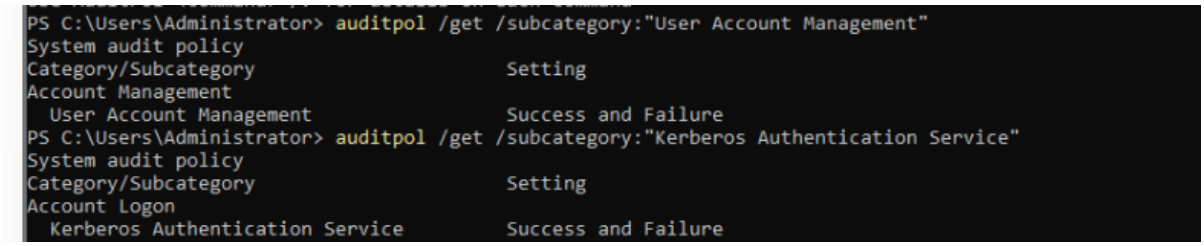


Figure 3. *User Account Management and Kerberos Authentication Service are audited for Success and Failure.*

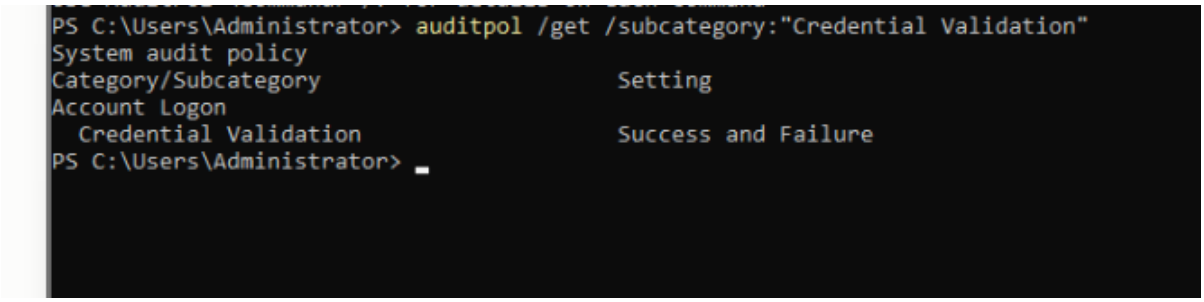


Figure 4. *Credential Validation is also audited for Success and Failure.*

Run on: DC01

```
wevtutil sl Security /ms:209715200
```

What it does: Increases the maximum size of the DC's Security log to 200 MB, so lockout evidence is not overwritten by newer events before I can investigate.

B4. Turn on failed sign-in auditing on the workstation

The domain controller can say **which computer** sent a bad password, but only the workstation's own log can say **what on that computer** did it. That information is in Event **4625**, which needs Logon auditing on the workstation.

Run on: WIN11-WS01 (signed in as .\Urvisha, run as Administrator)

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
```

What it does: Turns on auditing of successful and failed sign-ins on WIN11-WS01, which produces Event 4625 for failures.

Run on: WIN11-WS01

```
gpupdate /force
```

What it does: Pulls the latest Group Policy from DC01, including the new lockout policy.

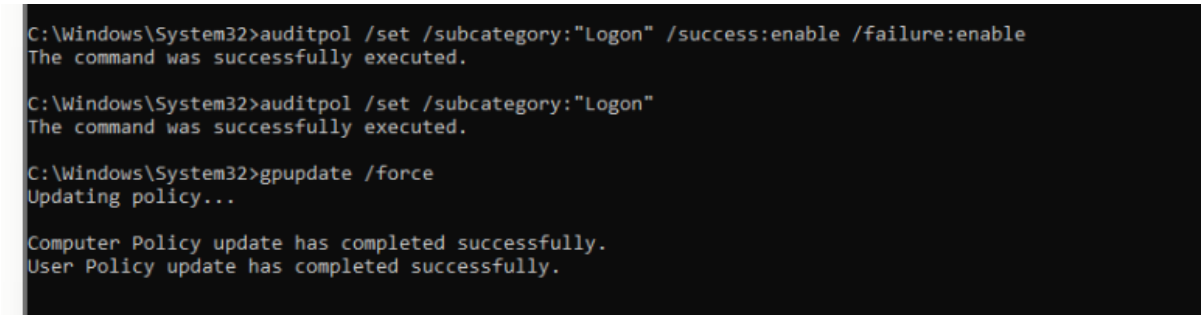


Figure 5. *Logon auditing enabled on WIN11-WS01 and Group Policy refreshed. (The second line was meant to be "auditpol /get" to confirm the setting.)*

Run on: WIN11-WS01

```
net accounts /domain
nltest /sc_verify:testuser.local
```

What it does: `nltest /sc_verify` tests the **secure channel**: the trusted connection between the workstation and the domain controller. **NERR_Success** means the computer's trust with the domain is healthy.

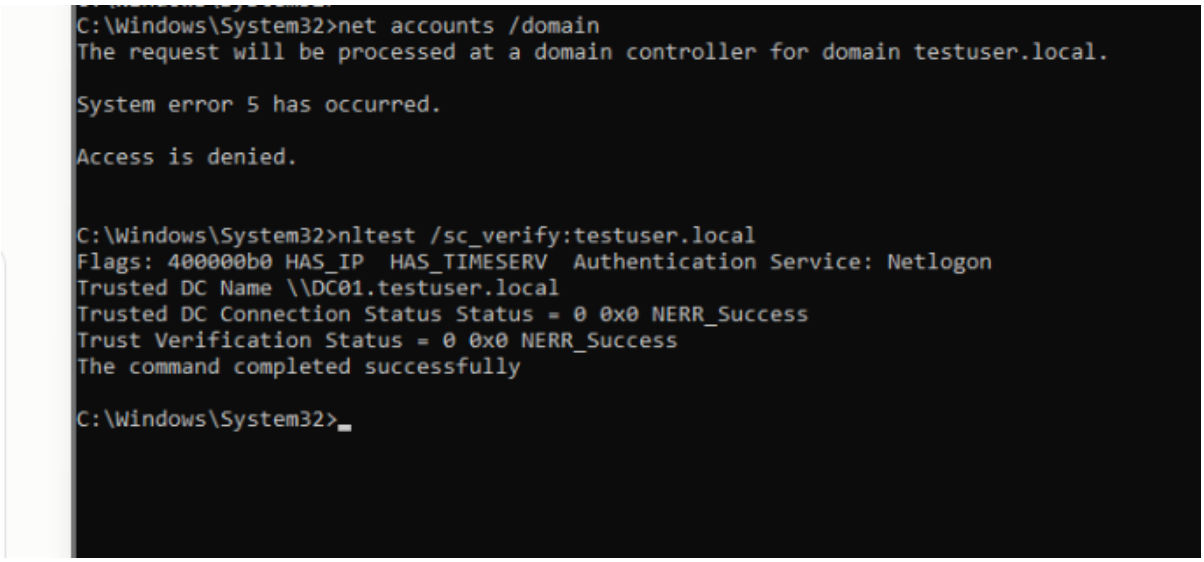


Figure 6. *nltest confirms a healthy secure channel to DC01.testuser.local. "System error 5 / Access is denied" for net accounts /domain is expected here: I was signed in as the local user Urvisha, who has no permission to read domain settings. The policy had already been confirmed on DC01.*

B5. Turn on Task Scheduler history

On WIN11-WS01 I opened **Task Scheduler as administrator** and clicked **Enable All Tasks History** in the Actions pane. This records every task start, success and failure, which later provided direct proof of the cause.

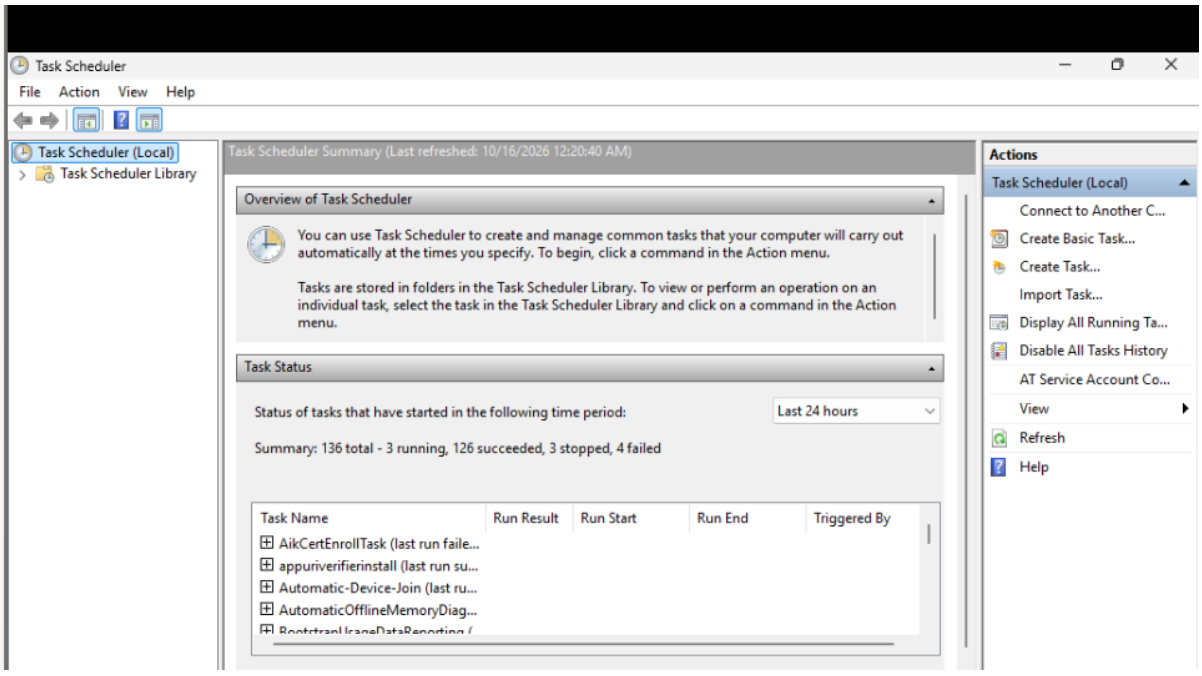


Figure 7. *Task Scheduler history is on: the Actions pane now offers "Disable All Tasks History".*

Part C: Planting the Problem

To practise a realistic investigation, I created the problem myself. In real companies this happens when someone sets up a script or backup job with their own account and forgets about it. After their next password change, the job keeps trying the old password.

C1. Create a scheduled task that runs as IT User01

I gave the task a harmless-looking name, "OneDrive Backup Sync". Problem tasks in real environments often look this innocent. Every minute it writes the date and time into C:\Temp\sync.log, so I could see whether it was working.

Run on: WIN11-WS01 (Windows PowerShell as Administrator)

```
New-Item -Path C:\Temp -ItemType Directory -Force
```

What it does: Creates the folder C:\Temp for the log file. `-Force` means no error if the folder already exists.

```
$action = New-ScheduledTaskAction -Execute "cmd.exe" -Argument "/c echo %date% %time% >>
C:\Temp\sync.log"
```

What it does: Defines **what** the task runs: a command that appends the current date and time to sync.log. It is stored in the variable `$action`.



Figure 8. *Defining the task action.*

```
$trigger = New-ScheduledTaskTrigger -Once -At (Get-Date).AddMinutes(1) -RepetitionInterval
(New-TimeSpan -Minutes 1)
```

What it does: Defines **when** the task runs: starting one minute from now and repeating every minute.

```
Register-ScheduledTask -TaskName "OneDrive Backup Sync" -Action $action -Trigger $trigger -
User "TESTUSER\ituser01" -Password "<IT User01 current password>"
```

What it does: Creates the task. The important part is `-User` and `-Password`: the task does not run as the person who created it (Urvisha), it **signs in as IT User01 with a saved copy of the password**. That saved copy is what later goes out of date.

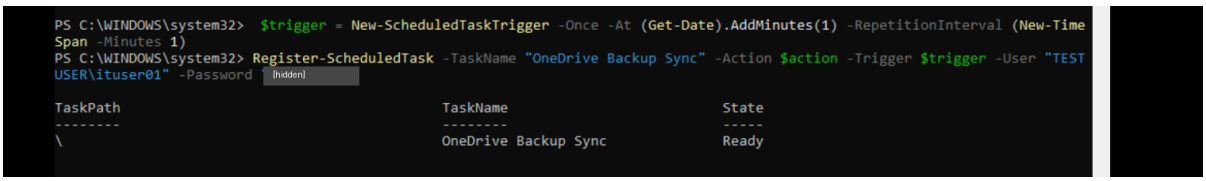


Figure 9. *The task is registered and Ready. The password in this screenshot has been hidden.*

PROBLEM I HIT: THE TASK NEVER STARTED

At first sync.log was never created. `Get-ScheduledTaskInfo` showed LastTaskResult **267011** (0x41303, "the task has not yet run") and a LastRunTime of 11/30/1999, which is Windows' placeholder for "never". The NextRunTime kept moving forward, which meant the task was trying to start and failing before it ran.

The cause: on Windows 11, standard domain users do not have the **"Log on as a batch job"** right, and a scheduled task with a saved password signs in as a batch job. I fixed it in **secpol.msc > Local Policies > User Rights Assignment > Log on as a batch job > Add User or Group > TESTUSER\ituser01**.

```
Start-ScheduledTask -TaskName "OneDrive Backup Sync"
Start-Sleep 5
Get-ScheduledTaskInfo -TaskName "OneDrive Backup Sync" | Select-Object LastRunTime,
LastTaskResult
```

What it does: `Start-ScheduledTask` runs the task immediately instead of waiting. `Start-Sleep 5` waits five seconds. `Get-ScheduledTaskInfo` shows when it last ran and the result code, where **0 means success**.

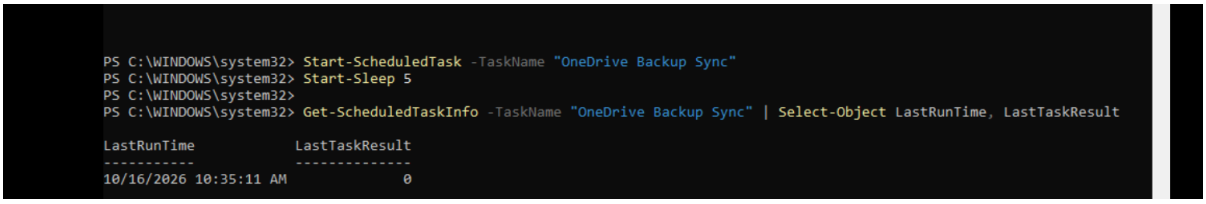


Figure 10. *After the fix, the task ran successfully (LastTaskResult 0).*

```
Get-Content C:\Temp\sync.log
```

What it does: Shows the contents of the log file. A new line every minute proves the task is signing in as IT User01 successfully.

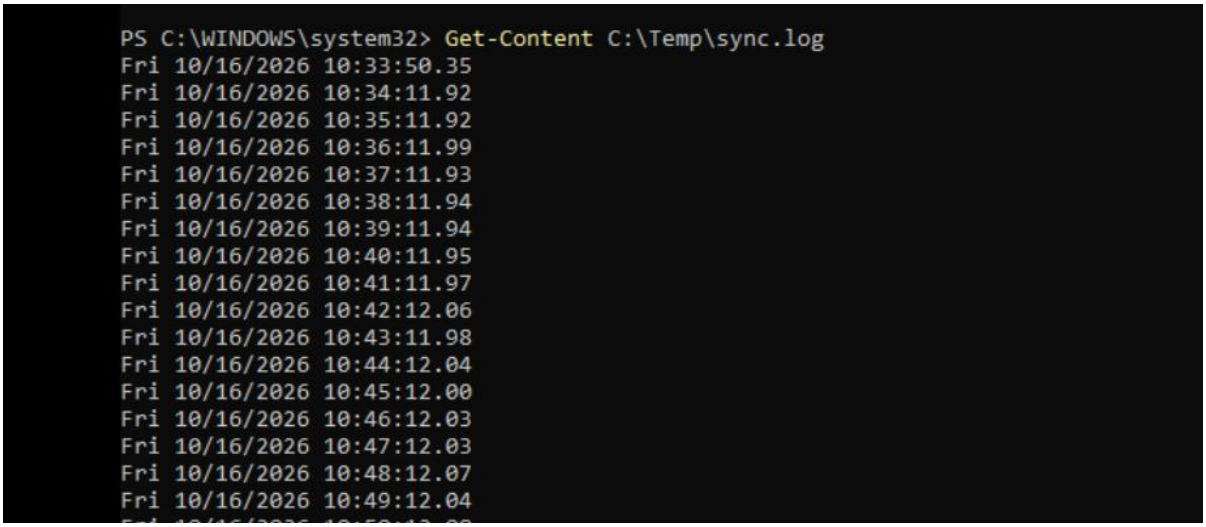


Figure 11. *sync.log gets a new line every minute: the task works with the current password.*

C2. Change IT User01's password three times

In **ADUC > Lab Users** I right-clicked **IT User01 > Reset Password** three times, each with a different password. The task still had the original password saved.

> **WHY THREE TIMES? (THE "N-2" RULE)**
>
> Active Directory does not count a wrong password as a bad attempt if it matches one of the user's **two most recent previous passwords**. This protects users who are still using a recently changed password somewhere.
>
> After only one or two changes, the task's password would fail every minute but **never lock** the account. After three changes, it is too old to be forgiven, so every failure counts toward the lockout. In real life, this explains why lockouts can seem to "start out of nowhere" weeks after the real cause appeared.

> **PROBLEM I HIT: CLOCKS DID NOT MATCH**
>
> DC01 showed the bad attempt at 11:56:11 AM while WIN11-WS01's task ran at 12:56:11 PM. The seconds matched, so it was the same event: DC01 was on a different time zone. I set DC01 to Mountain Time (`Get-TimeZone`, `Set-TimeZone -Id "Mountain Standard Time"`) so both machines' logs line up. Matching clocks matter in any log investigation.

Run on: DC01

```
Get-ADUser ituser01 -Properties LockedOut, BadLogonCount, LastBadPasswordAttempt | Select-Object SamAccountName, LockedOut, BadLogonCount, LastBadPasswordAttempt
```

What it does: Shows the lockout state, the number of bad attempts and the time of the last bad attempt. I ran it about once a minute to watch the count rise.

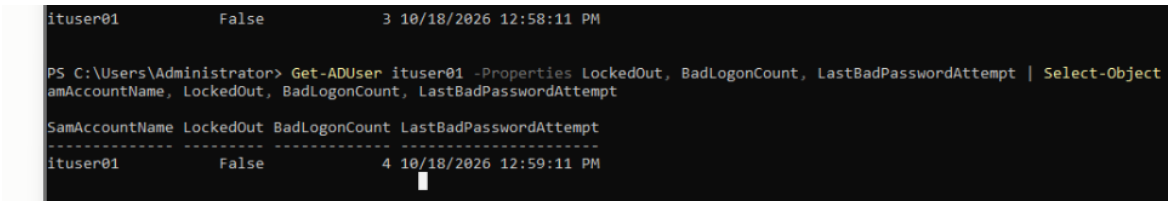


Figure 12. *BadLogonCount climbs by one each minute (3 at 12:58, 4 at 12:59): one failed attempt every time the task runs.*

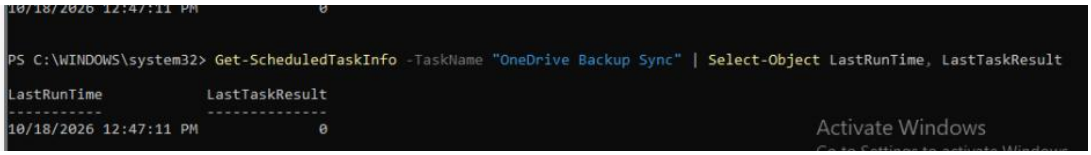


Figure 13. *A trap: Task Scheduler still shows the last successful run (12:47 PM, result 0). When the sign-in fails before the task starts, Windows does not record a new run, so the task looks healthy even though it fails every minute.*

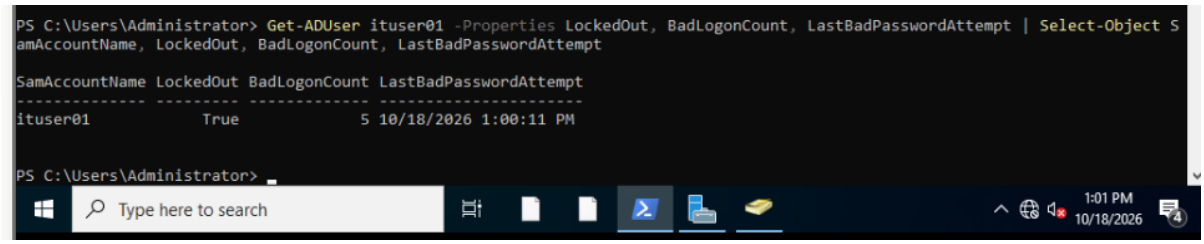


Figure 14. *At 1:00:11 PM the fifth bad attempt locked the account (LockedOut True, BadLogonCount 5).*

Part D: The Investigation

From here I worked as the help desk technician who only has the ticket. The goal was to answer two questions: **which computer** is sending the bad password, and **what on that computer** is doing it.

D1. Confirm the symptom

I tried to sign in to WIN11-WS01 as TESTUSER\ituser01 with the **correct, current** password.

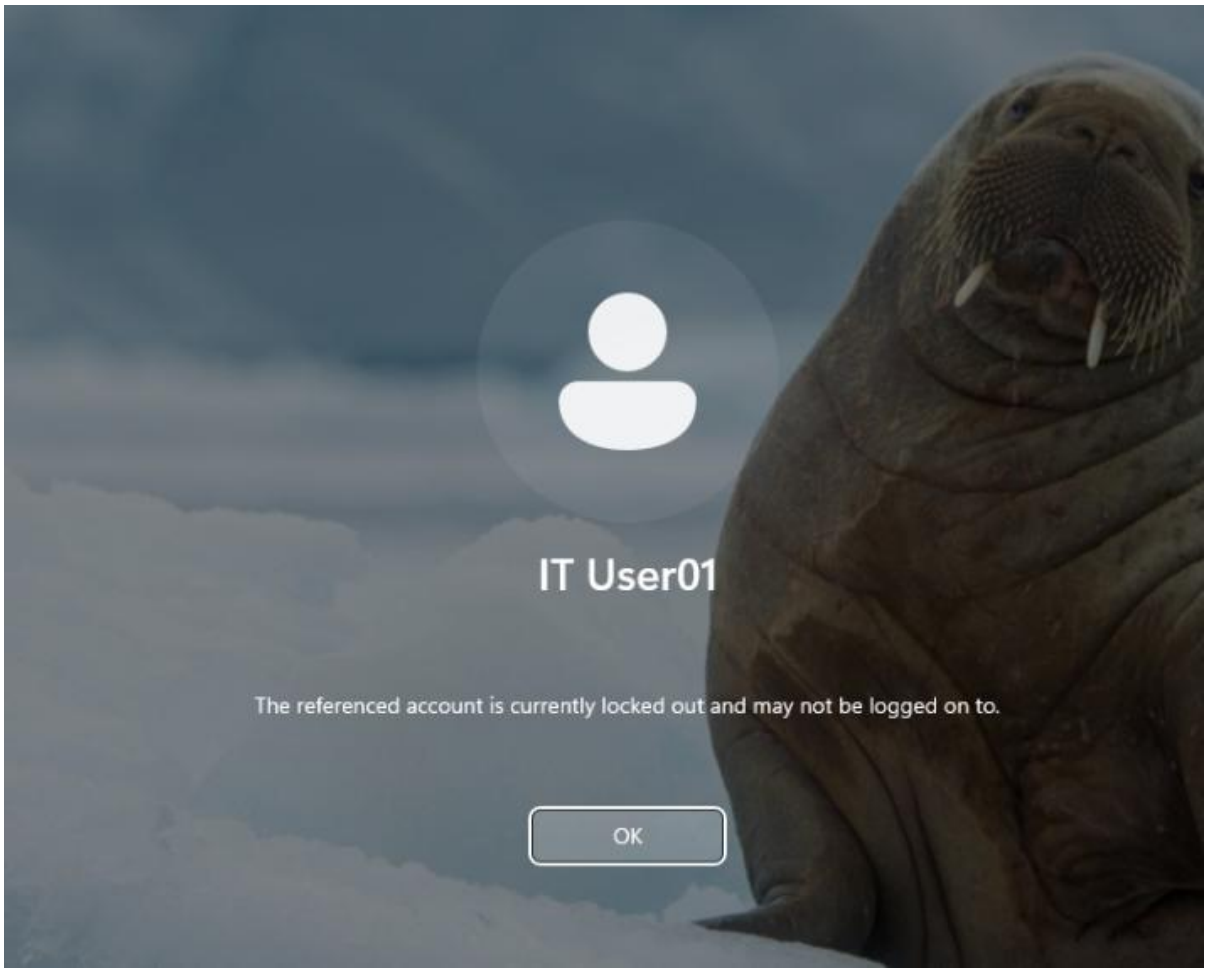


Figure 15. *The sign-in fails with "The referenced account is currently locked out and may not be logged on to", even with the correct password.*

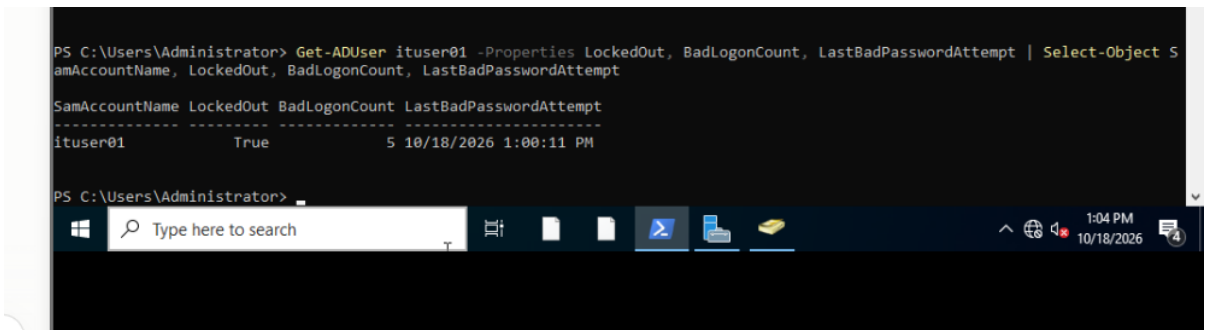


Figure 16. *DC01 confirms the account is locked (LockedOut True, BadLogonCount 5).*

D2. Show that unlocking alone does not fix it

In **ADUC > IT User01 > Properties > Account**, I ticked **Unlock account** and clicked OK.

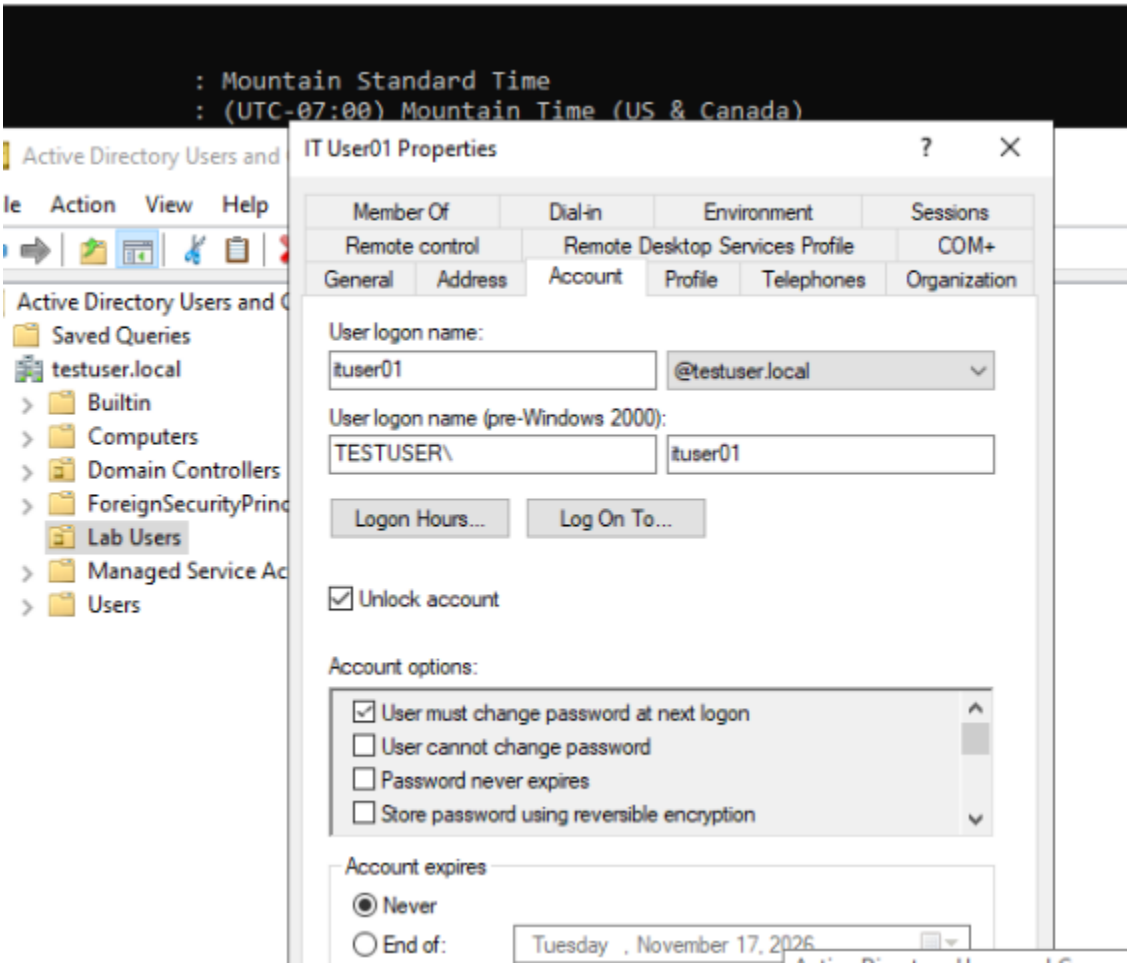


Figure 17. *Unlocking the account in Active Directory Users and Computers.*

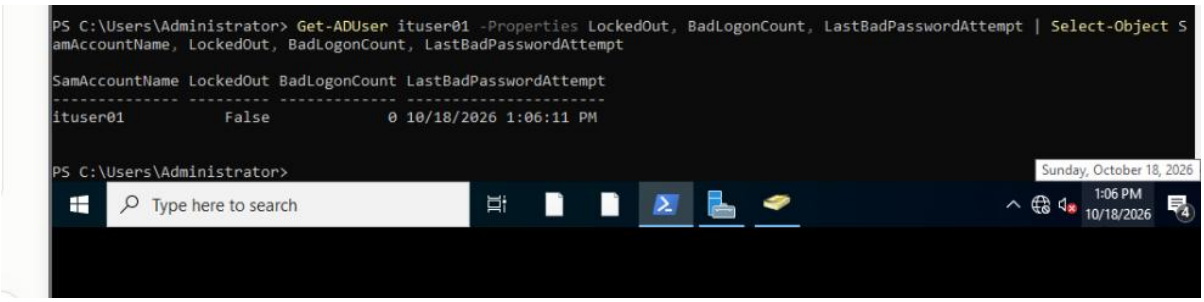


Figure 18. *Immediately after unlocking: LockedOut False.*

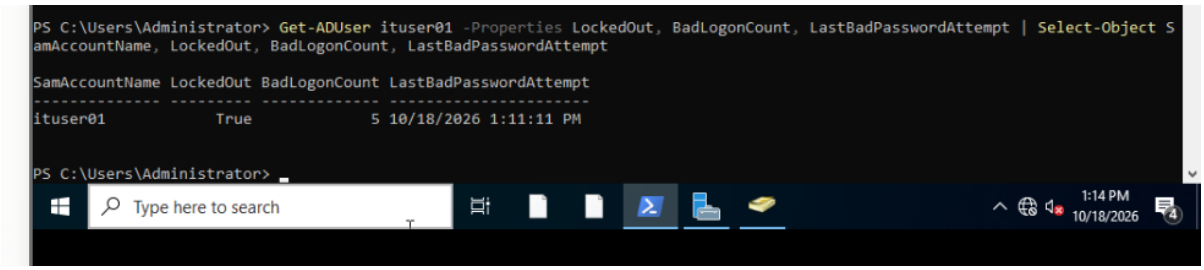


Figure 19. *Five minutes later the account was locked again (1:11:11 PM). Unlocking only treated the symptom; something was still sending the old password.*

D3. Find which computer is causing it (Event 4740)

On DC01 (the PDC Emulator) I opened **Event Viewer > Windows Logs > Security > Filter Current Log** and entered Event ID **4740**.

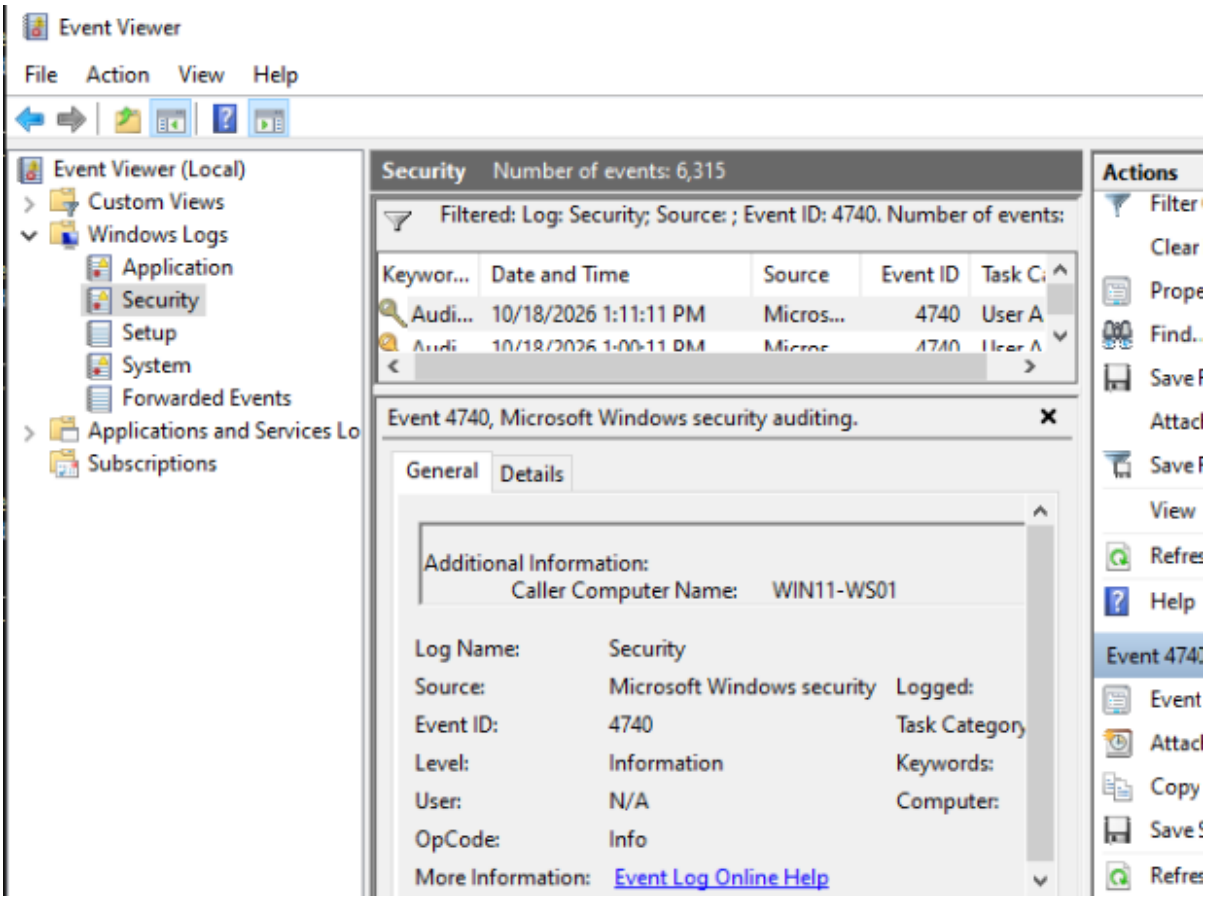


Figure 20. *Both lockout events (1:00 PM and 1:11 PM) show Caller Computer Name: WIN11-WS01. The bad passwords were coming from that computer, not from the user typing.*

I also filtered for Event **4771** (failed Kerberos sign-in). The **Client Address** was ::ffff:192.168.76.137, which is WIN11-WS01 (`::ffff:` is how Windows writes an IPv4 address in IPv6 form). The failure codes told the story:

Failure code	Meaning	What it showed
0x18	Wrong password	The attempts that built up to the lockout

Failure code	Meaning	What it showed
0x12	Account locked or disabled	Attempts after the lockout: the source kept trying even while the account was locked

D4. Find what on that computer is using the old password (Event 4625)

On WIN11-WS01 I opened **Event Viewer > Security** and filtered for Event **4625** (failed sign-in on this computer).

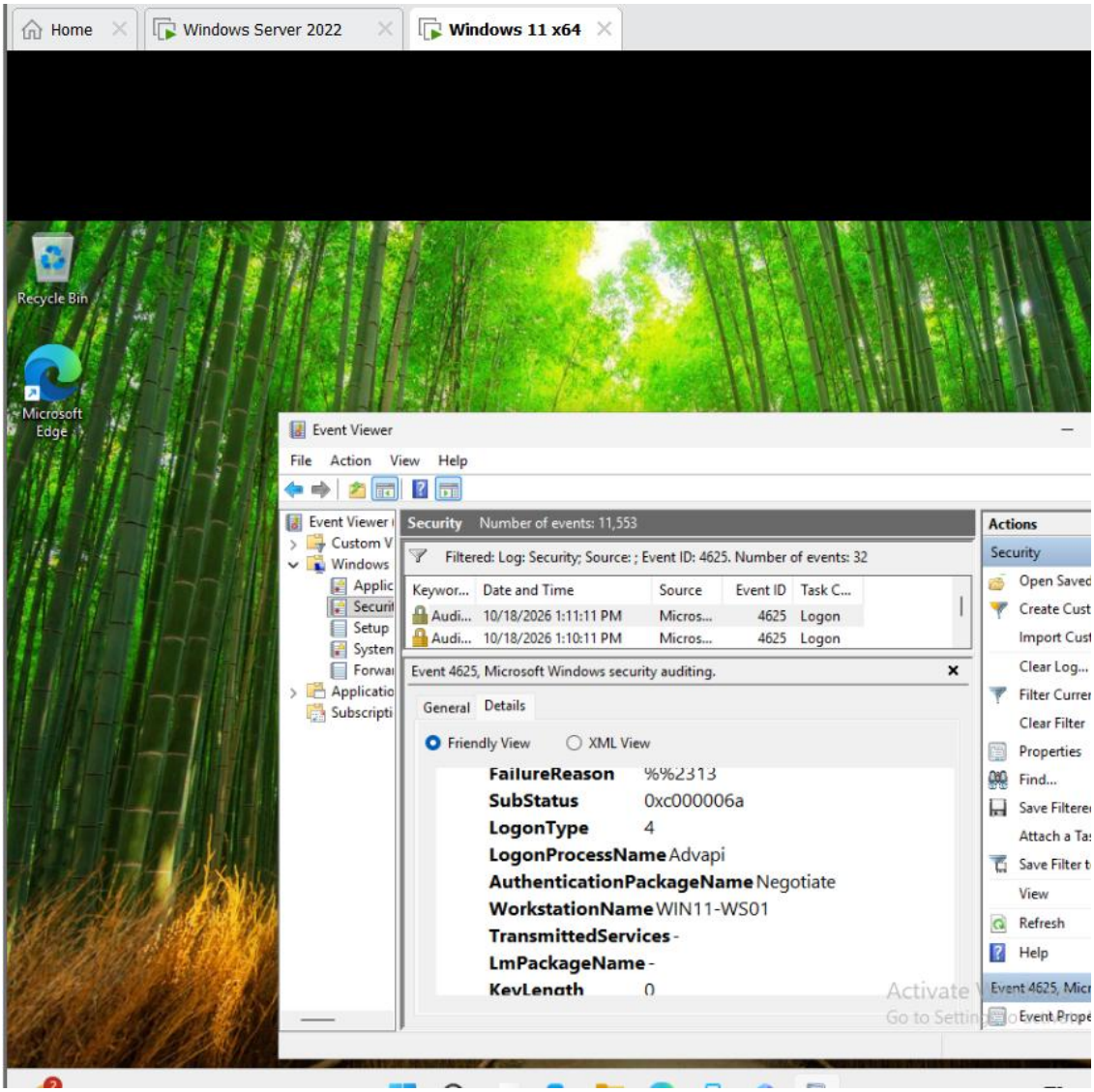


Figure 21. *Event 4625 on WIN11-WS01: Logon Type 4 and Sub Status 0xC000006A (wrong password).*

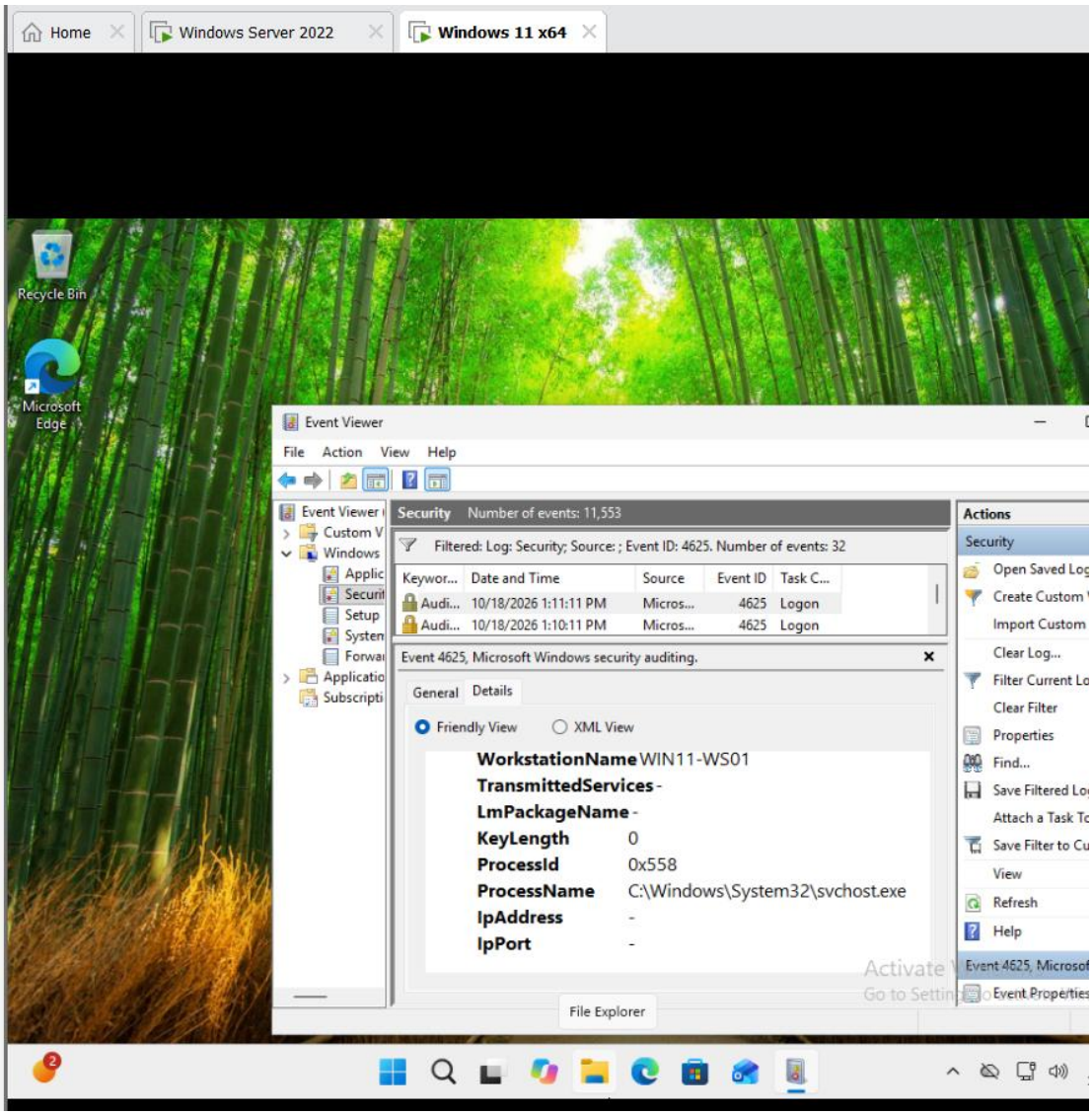


Figure 22. *The same event: the failing process was C:\Windows\System32\svchost.exe, the process that hosts the Task Scheduler service.*

The **Logon Type** says what kind of sign-in failed:

Logon Type	Meaning	Typical source
2	Interactive	Someone typing at the keyboard
3	Network	Mapped drive or shared folder
4	**Batch**	**Scheduled task** (the type seen here)
5	Service	A Windows service running as the user
10	RemoteInteractive	Remote Desktop

D5. Identify the exact task

Run on: WIN11-WS01

```
Get-ScheduledTask | Where-Object { $_.Principal.UserId -match "ituser01" } | Select-Object
TaskName, TaskPath, State
```

What it does: Lists every scheduled task, keeps only the ones that run as ituser01, and shows their names and state.

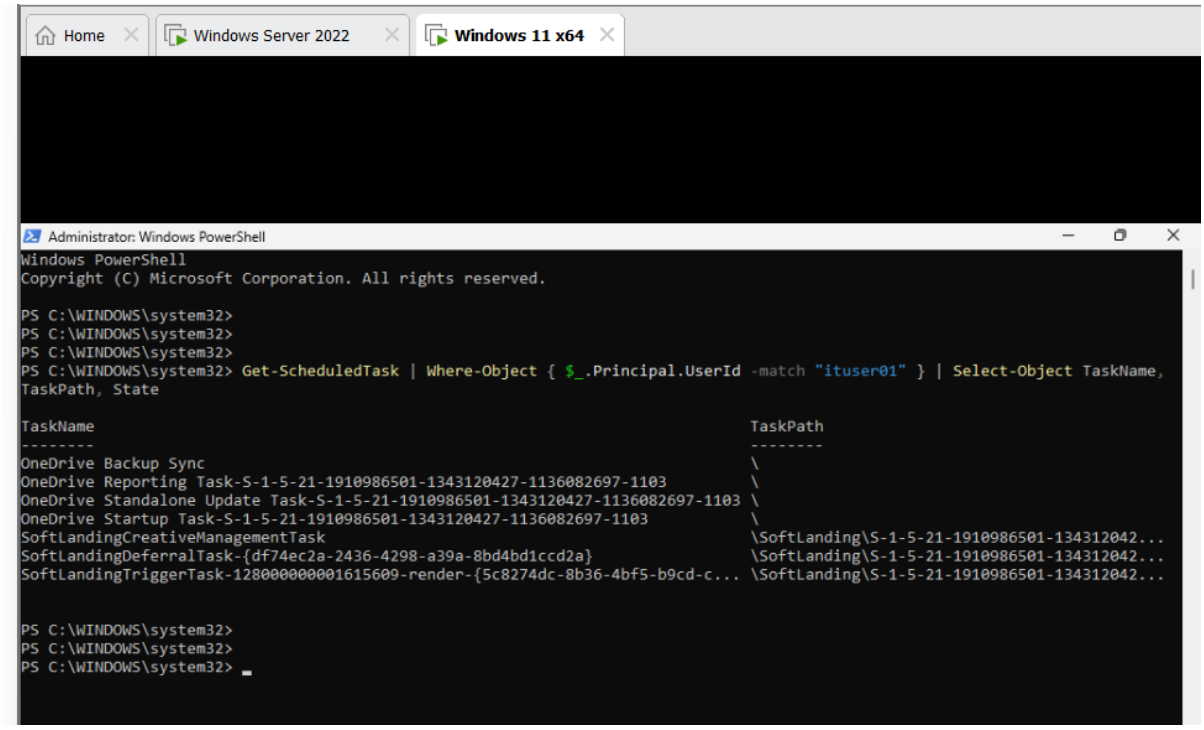


Figure 23. Seven tasks are linked to IT User01, so I needed one more check to find the one responsible.

To be thorough, I also ruled out the other common lockout sources:

Run on: WIN11-WS01

```
Get-CimInstance Win32_Service | Where-Object StartName -match "ituser01" | Select-Object Name,
StartName
cmdkey /list
net use
```

What it does: `Get-CimInstance Win32_Service` lists Windows services running as ituser01. `cmdkey /list` lists passwords saved in Credential Manager. `net use` lists mapped network drives.

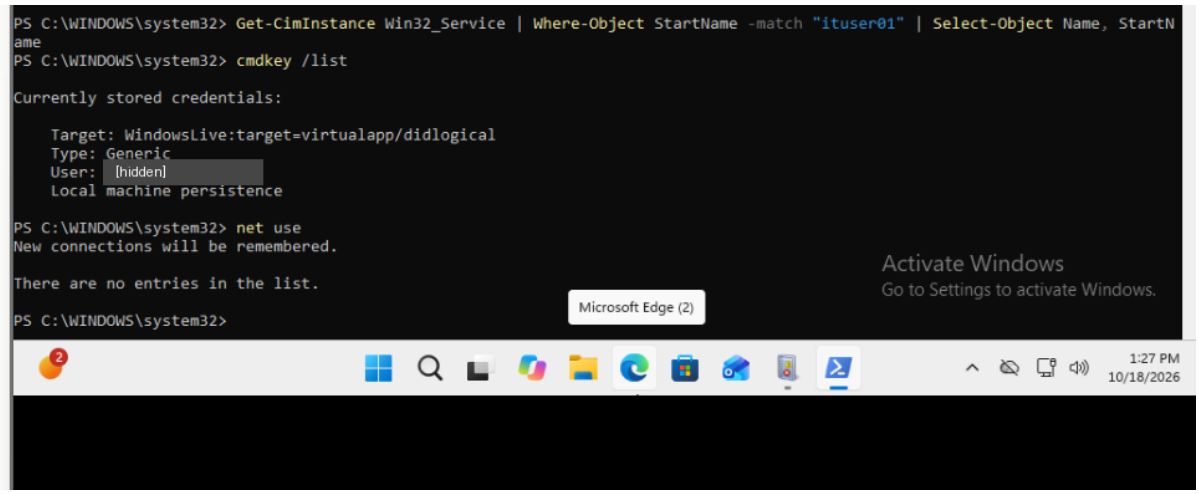


Figure 24. *No services, no mapped drives, and no saved credential for ituser01 (the one entry is a built-in Microsoft account token, hidden here).*

> **LIMIT OF THIS CHECK**
>
> `cmdkey /list` only shows credentials for the account that runs it (here Urvisha). It cannot see IT User01's own Credential Manager. In a real ticket I would also check Credential Manager while signed in as the user, or ask the user.

Run on: WIN11-WS01

```
Get-ScheduledTask | Where-Object { $_.Principal.UserId -match "ituser01" } | Select-Object TaskName, @{n='LogonType';e={$_.Principal.LogonType}}, State
```

What it does: Adds the **LogonType** column. A task can only send a wrong password to the DC if it **stores a password** (LogonType: Password). Tasks with LogonType Interactive only run inside an existing sign-in and never send a saved password.

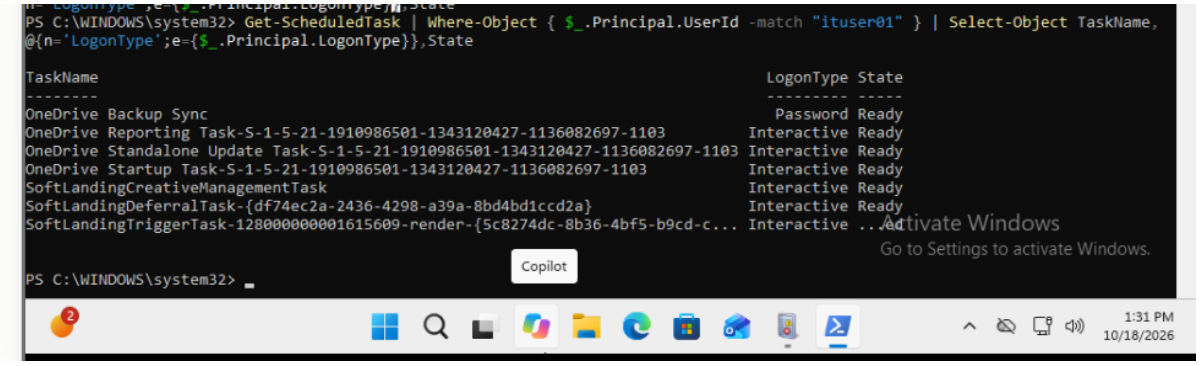


Figure 25. *Only "OneDrive Backup Sync" uses a saved password. The other six are Windows' built-in per-user tasks (OneDrive and SoftLanding), so they were left alone.*

CONFIRMED CAUSE

The scheduled task **"OneDrive Backup Sync"** on **WIN11-WS01** runs as **TESTUSER\ituser01** with a **saved password that is three changes old**.

Evidence: Event 4740 (Caller Computer = WIN11-WS01), Event 4771 (client 192.168.76.137, code 0x18), Event 4625 (Logon Type 4, svchost.exe), LogonType = Password, and the task's history showing failed starts.

Part E: The Fix

E1. Fix in the right order

If the account is unlocked first, the task locks it again within minutes, which is exactly what happened at 1:11 PM. So the source must be stopped **before** unlocking.

Step 1, run on: WIN11-WS01

```
Disable-ScheduledTask -TaskName "OneDrive Backup Sync"
```

What it does: Turns the task off so it stops sending the old password. This is a safe, reversible change: the task is kept, just not run.

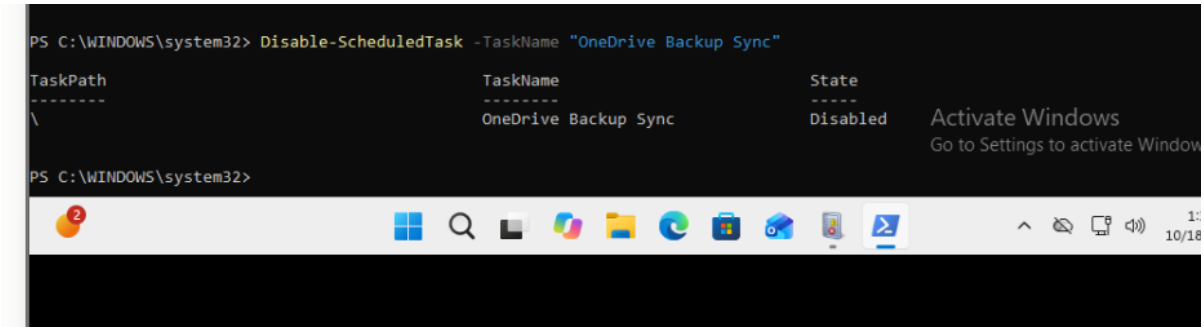


Figure 26. *Step 1: the source is stopped (State: Disabled).*

Step 2: On DC01, ADUC > IT User01 > Properties > Account > **Unlock account** > OK.

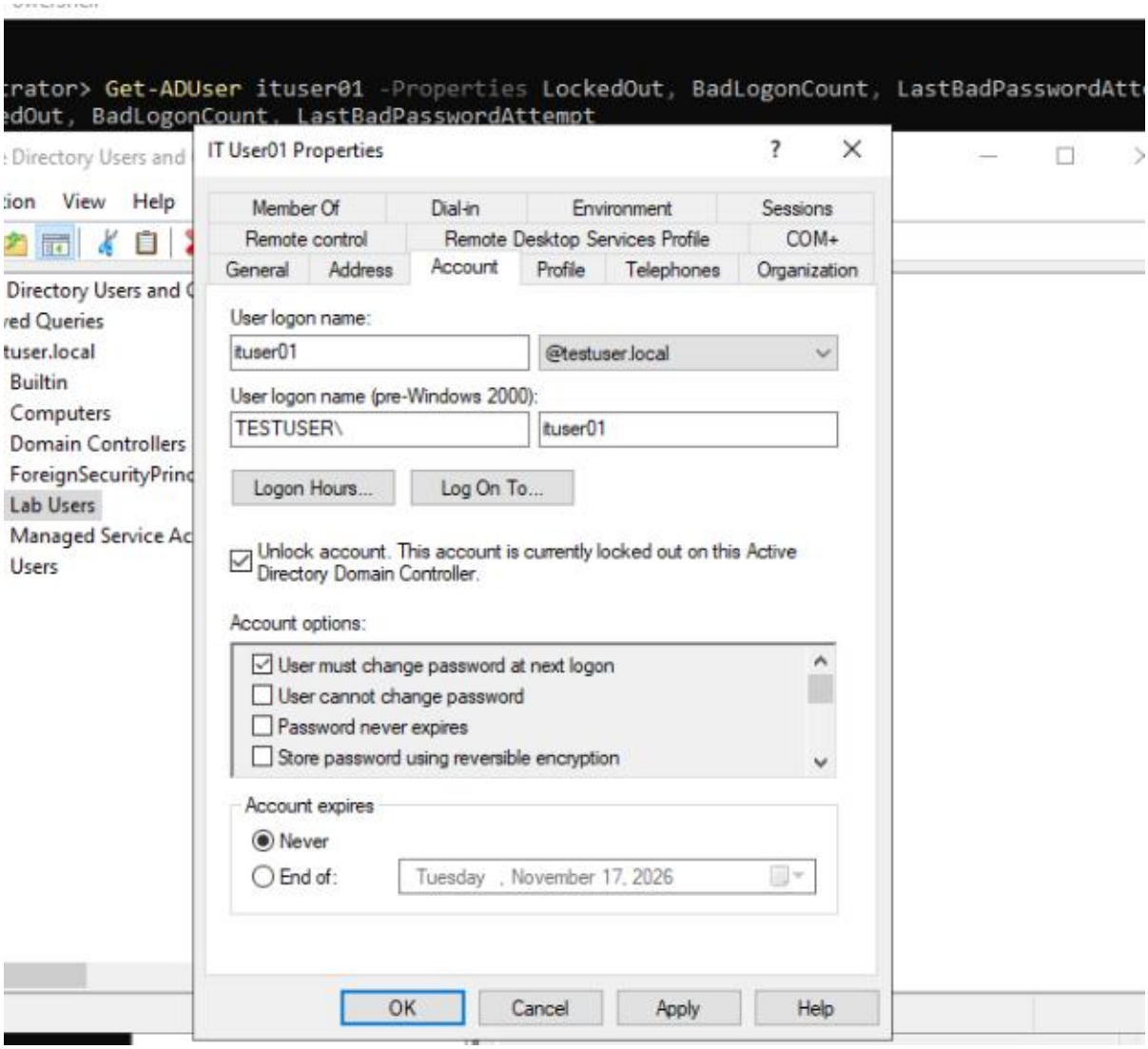


Figure 27. *Step 2: unlocking the account. Note that "User must change password at next logon" is ticked. This caused the next problem.*

Step 3: Save the current password in the task:

Run on: WIN11-WS01

```
Set-ScheduledTask -TaskName "OneDrive Backup Sync" -User "TESTUSER\ituser01" -Password
"<current password>"
```

What it does: Updates the credentials stored in the existing task. Windows checks the password when saving it, which is why this step comes after the unlock.

PROBLEM I HIT: 0x80070773

The first attempt failed with "The user's password must be changed before signing in" (HRESULT 0x80070773). The account had **"User must change password at next logon"** ticked, which ADUC's Reset Password window ticks by default. A scheduled task has no screen, so it cannot answer a password-change prompt.

Fix on DC01: `Set-ADUser ituser01 -ChangePasswordAtLogon $false`. This unticks the box (behind the scenes it changes the pwdLastSet attribute from 0 to the current time). After that, Set-ScheduledTask succeeded.

In production, the user would sign in and choose their own new password first, then the task would be updated with it.

Step 4: Turn the task back on and test it:

Run on: WIN11-WS01

```
Enable-ScheduledTask -TaskName "OneDrive Backup Sync"
Start-ScheduledTask -TaskName "OneDrive Backup Sync"
Get-Content C:\Temp\sync.log -Tail 3
Get-ScheduledTaskInfo -TaskName "OneDrive Backup Sync" | Select-Object LastRunTime,
LastTaskResult, NextRunTime
```

What it does: `Enable-ScheduledTask` turns the task back on. `Get-Content -Tail 3` shows only the last three lines of the log.

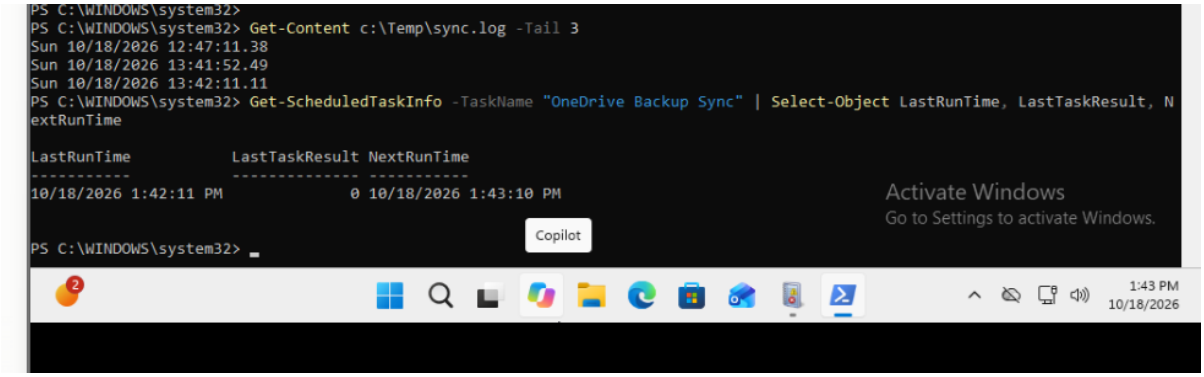


Figure 28. The fix works: after a gap from 12:47 to 13:41 (the outage), the task runs again (manual test at 13:41:52, scheduled run at 13:42:11) with LastTaskResult 0.

CHANGE MADE

Disabled the task, unlocked IT User01, cleared "must change password at next logon", saved the current password in the task, then re-enabled and tested it.

In a company I would first confirm with the task's owner that it is still needed, and recommend moving it to a dedicated service account so a user's password change cannot break it.

E2. Final verification

I waited more than ten minutes and checked everything again:

Check	Result
Get-ADUser on DC01	LockedOut **False**, BadLogonCount **0**, last bad attempt still **1:30:11 PM** (no new failures after the fix)
Event 4740 on DC01	No new lockouts after the fix
sync.log on WIN11-WS01	A new line every minute since 13:42
Sign in as TESTUSER\ituser01	Signed in normally

FINAL RESULT

IT User01 is no longer being locked out. The task runs successfully with the current password, and no new bad password attempts or lockout events have been recorded since the fix.

Part F: Automating the Investigation

Finding the source took several minutes of clicking in Event Viewer. I wrote **Find-LockoutSource.ps1** so the same investigation runs in one command. It is included with this report.

Section	What the script does
1. Account status	Reads LockedOut, BadLogonCount, LastBadPasswordAttempt, AccountLockoutTime and PasswordLastSet from the PDC Emulator
2. Lockout events	Reads Event 4740 for the user and lists each lockout time with its Caller Computer
3. Failed sign-ins	Reads Event 4771 for the user and groups the failures by client IP and failure code (0x18 wrong password, 0x12 locked)
Next step	Reminds the technician to check Event 4625 and the Logon Type on the source computer

Run on: DC01, in C:\Scripts

```
Set-ExecutionPolicy RemoteSigned -Scope Process -Force
.\Find-LockoutSource.ps1 -User ituser01 -Hours 24
```

What it does: The first line allows scripts to run **in this PowerShell window only** (it does not change the system-wide setting). The second runs the script for ituser01 over the last 24 hours.

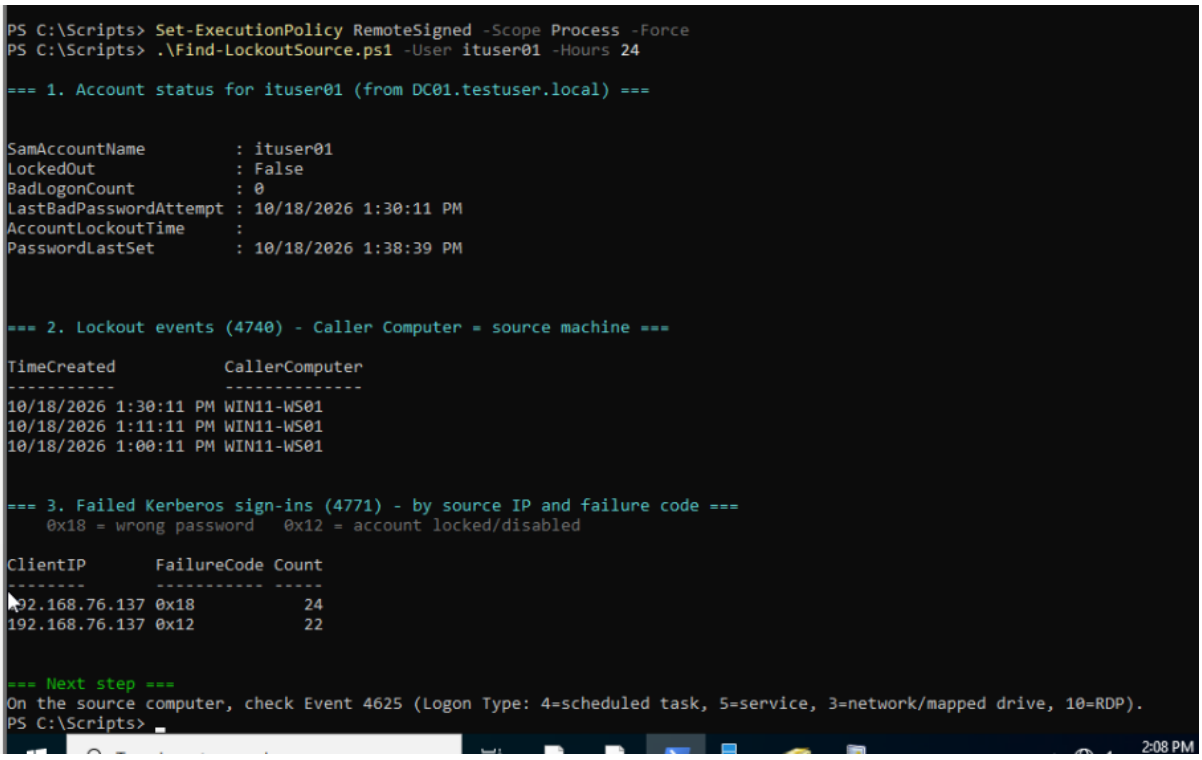


Figure 29. *The script summarises the incident in seconds: three lockouts, all from WIN11-WS01, and 46 failed Kerberos sign-ins from 192.168.76.137 (24 wrong password, 22 already locked).*

The script output also revealed a **third lockout at 1:30 PM**. After the 1:11 lockout, the account unlocked itself when the 15-minute lockout duration expired, and the task locked it again a few minutes later. Waiting did not help either; only fixing the source did.

Incident Timeline

Date / time	Event
Oct 16, 10:33 AM	Scheduled task created and confirmed working (sync.log updating every minute)
Oct 18, 12:47 PM	Last successful task run before the password changes
Oct 18, ~12:50 PM	IT User01's password reset three times on DC01
Oct 18, 12:58 to 12:59 PM	BadLogonCount rises (3, then 4)
Oct 18, 1:00:11 PM	**First lockout** (Event 4740, caller WIN11-WS01)
Oct 18, ~1:06 PM	Account unlocked by the help desk
Oct 18, 1:11:11 PM	**Second lockout**: unlocking did not fix the problem
Oct 18, 1:30:11 PM	**Third lockout** after the automatic 15-minute unlock
Oct 18, 1:27 to 1:31 PM	Investigation on WIN11-WS01: other sources ruled out, task identified by LogonType
Oct 18, ~1:35 PM	Task disabled (source stopped) and account unlocked
Oct 18, 1:38 PM	"Must change password" flag cleared; task password updated
Oct 18, 1:42 PM	First successful scheduled run after the fix
Oct 18, 2:08 PM	Find-LockoutSource.ps1 confirms no new failures since 1:30 PM

Command Reference

Every command used in this project, where it was run and what it does.

Command	Where	Purpose
`netsh interface ipv4 set address ...`	DC01	Set a static IP, mask and gateway
`netsh interface ipv4 set dnsservers ...`	DC01	Point DNS to the DC itself
`ipconfig /all`	Both	Show full network settings
`Rename-Computer -NewName DC01 -Restart`	DC01	Rename the server and restart
`Get-ADDomain`	DC01	Show domain name and the PDC Emulator
`Get-ADUser ... -Properties LockedOut, BadLogonCount ...`	DC01	Check lock state and bad password count
`net accounts /domain`	DC01	Show the domain password and lockout policy
`gpupdate /force`	Both	Apply Group Policy immediately
`auditpol /get` and `auditpol /set`	Both	Check or set audit policy
`wevtutil sl Security /ms:209715200`	DC01	Enlarge the Security log to 200 MB
`nltest /sc_verify:testuser.local`	WIN11-WS01	Test the secure channel (trust) with the domain
`Get-TimeZone` / `Set-TimeZone`	DC01	Check and match time zones for log comparison

Command	Where	Purpose
`New-Item -ItemType Directory`	Both	Create a folder
`New-ScheduledTaskAction`	WIN11-WS01	Define what a task runs
`New-ScheduledTaskTrigger`	WIN11-WS01	Define when a task runs
`Register-ScheduledTask`	WIN11-WS01	Create a task that runs as a given user
`Start-ScheduledTask`	WIN11-WS01	Run a task immediately
`Get-ScheduledTaskInfo`	WIN11-WS01	Show last run time and result code
`Get-Content ... -Tail 3`	WIN11-WS01	Show the last lines of a file
`Get-WinEvent -LogName Microsoft-Windows-TaskScheduler/Operational`	WIN11-WS01	Read Task Scheduler's own event log (Event 101 = failed start)
`Get-ScheduledTask | Where-Object ...`	WIN11-WS01	Find tasks running as a user, and their LogonType
`Get-CimInstance Win32_Service`	WIN11-WS01	Find services running as a user
`cmdkey /list`	WIN11-WS01	List saved credentials (current user only)
`net use`	WIN11-WS01	List mapped network drives
`Disable-ScheduledTask` / `Enable-ScheduledTask`	WIN11-WS01	Stop or restart a task safely
`Set-ScheduledTask -User -Password`	WIN11-WS01	Update the password saved in a task
`Set-ADUser -ChangePasswordAtLogon $false`	DC01	Clear "must change password at next logon"
`Unlock-ADAccount`	DC01	Unlock an account from PowerShell (same as the ADUC checkbox)
`Set-ExecutionPolicy RemoteSigned -Scope Process`	DC01	Allow scripts in the current window only
`.\Find-LockoutSource.ps1 -User ituser01`	DC01	Run the lockout investigation script

Graphical tools used

Tool	How to open	Used for
Server Manager	Opens at sign-in on the server	Installing AD DS and promoting the DC
Active Directory Users and Computers	Server Manager > Tools	Creating the user, resetting passwords, unlocking
Group Policy Management	Server Manager > Tools	Lockout policy and audit policy
Event Viewer	Win + R > eventvwr.msc	Events 4740, 4771, 4625
Task Scheduler	Start > Task Scheduler (as admin)	Task history and properties
Local Security Policy	Win + R > secpol.msc	Granting "Log on as a batch job"
Network Connections	Win + R > ncpa.cpl	Static IP and DNS on the workstation

Tool	How to open	Used for
System Properties	Win + R > sysdm.cpl	Renaming the PC and joining the domain

What I Learned

- **Unlocking is not a fix.** The account locked again at 1:11 PM and 1:30 PM until the source was stopped. Always fix the source first, then unlock.
- **Event 4740 on the PDC Emulator** tells you which computer caused a lockout (Caller Computer Name). That is always the first place to look.
- **Event 4625 Logon Type** tells you what kind of thing failed on that computer: 4 = scheduled task, 5 = service, 3 = network, 10 = RDP.
- **The "n-2" rule:** Active Directory forgives the two most recent previous passwords, so a stale credential may not cause lockouts until after another password change.
- **Task Scheduler can look healthy while failing:** when the sign-in fails before the task starts, Last Run Result still shows the last success. Check event logs, not just that column.
- **LogonType: Password vs Interactive** separates tasks that store a password (possible culprits) from built-in per-user tasks (harmless). This avoided deleting Windows' own OneDrive tasks.
- **"User must change password at next logon"** blocks scheduled tasks and services, because they cannot answer the prompt (error 0x80070773).
- **"Log on as a batch job"** is required for a task that runs with a saved password; without it the task never starts.
- **Matching time zones** across systems is essential when comparing logs from different machines.
- **PowerShell vs Command Prompt:** commands such as `$action = ...` only work in PowerShell (prompt starts with PS).

Summary and Security Recommendations

A user was locked out repeatedly even though they typed the correct password. Using the domain controller's lockout events, I traced the bad passwords to WIN11-WS01. Using the workstation's failed sign-in events, I narrowed the cause to a scheduled task, then confirmed the exact task by its logon type. I stopped the task, unlocked the account, updated the task's saved password, and verified that the lockouts stopped. I also built a PowerShell script that performs the investigation in one command.

Recommendations

- **Do not run scheduled tasks or services under personal user accounts.** Use dedicated service accounts or, better, **group Managed Service Accounts (gMSA)**, whose passwords Windows rotates automatically.
- **After a password change, remind users** to update saved passwords on phones, other PCs, mapped drives and apps.
- **Keep lockout and logon auditing enabled** on domain controllers and workstations, and size the Security log so evidence is kept long enough.
- **Forward security events to a central place (SIEM)** so lockout sources can be found without signing in to each machine. **Watch for many lockouts across many users at once.** That pattern points to a password-spray attack rather than a stale credential and should be escalated to security.